Prototype-Initialized Adaptation of ViT-B/16 for 10,000-Class Fine-Grained Classification under a Fixed Compute Budget

Member 1 (student ID) Member 2 (student ID) Member 3 (student ID)
CS604 Team 06, Singapore Management University
email addresses

Abstract

We adapt a ViT-B/16 pretrained on ImageNet-21k and fine-tuned on ImageNet-1k to an anonymized fine-grained benchmark with 10,000 classes and only 45 labeled images per class, under a budget of 2 hours on 48 GB of GPU memory per training run. We compare seven adaptation methods under one protocol, from a linear probe and partial fine-tuning to BitFit, Adapter, AdaptFormer and LoRA, and find that the most important design decision is how the new 10,000-way classifier head is initialized. Initializing it with a linear probe fitted on frozen training features, a common way to avoid a random head, consistently hurts. Imprinting L2-normalized class-mean prototypes instead improves holdout accuracy by 2.1–2.5 points for full fine-tuning, LoRA and AdaptFormer, and the validation accuracy of our final model by 3.1 points under otherwise identical settings. The comparison also shows that where the trainable parameters sit matters more than how many there are: LoRA on 1.375% of the backbone matches fine-tuning its last four blocks (33%), and training memory follows how far gradients propagate, not the number of trained parameters. Together with training on all images and horizontal-flip test-time augmentation, our final full fine-tuning model reaches 76.22% top-1 accuracy on the CodaBench validation set (reference: 70.8%) using 1.03 GPU-hours on one A100. Our best parameter-efficient model applies LoRA to every linear layer of the backbone, trains 1.375% of its parameters, and reaches 71.90% (reference LoRA: 68.3%) with no inference overhead after the low-rank updates are merged.

1. Introduction

Large pretrained Vision Transformers [5, 19] are rarely trained from scratch; they are adapted to new tasks, either by fine-tuning all weights or by training a small set of added parameters [3, 6, 7]. In this project we adapt ViT-B/16 to an anonymized fine-grained classification benchmark with 10,000 classes. The training split has 450,000 images, exactly 45 per class; the validation split (50,000 images, 5 per class) and the test split (100,000 images, 10 per class) have hidden labels and are scored on CodaBench. Each training run may use at most 2 hours on 48 GB of GPU memory (96 GB·h), and every method is reported with four metrics: top-1 accuracy, trainable parameters, inference latency and training GPU-hours.

This setting differs from typical transfer-learning benchmarks in two ways. First, the classifier head is large and data-starved. A $10{,}000 \times 768$ linear head has 7.69M parameters: 9.0% of the 85.8M-parameter backbone and 26× more than a rank-8 LoRA on the attention projections. Yet each of its rows is trained from only 43–45 images. Second, the budget is tight. One epoch of full fine-tuning takes about 6.5 minutes on an A100, so only about eight epochs fit into a run.

These constraints raise three practical questions. (1) How should the new head be initialized? (2) Which adaptation method should we use, and where should its trainable parameters go? (3) How should a fixed budget be split between epochs, data and test-time computation? We answer them with 26 controlled runs, each changing one setting at a time. We select models on a local holdout split that agrees with the CodaBench validation score to within 0.3 points. Our contributions are:

- **Prototype head initialization.** The common linear-probe (LP) head, which LP-FT [12] fits on frozen features before fine-tuning, is harmful in this regime. Imprinting class-mean prototypes [17] instead improves accuracy by 2.1–3.1 points for full fine-tuning, LoRA and AdaptFormer. Loss curves suggest why (Sec. 4.5).
- **A controlled comparison of seven adaptation methods.** With one protocol, methods that make small changes in every block are far more parameter-efficient than fine-tuning the top blocks: LoRA on all linear layers (1.375% of the backbone) matches the last four blocks (33%), and BitFit (0.12%) beats the last block (8.3%). LoRA is the most accurate parameter-efficient method and reaches 98% of full fine-tuning accuracy; merged, it adds no latency, whereas the two adapters add 33–40%. Training memory follows

how far gradients propagate rather than how many parameters are trained, so parameter-efficient training is not automatically light (Sec. 4.4).

- **A budget-aware recipe with complete accounting.** We report accuracy, trainable parameters, latency, GPU-hours and peak memory for every run. Our final models reach 76.22% (full fine-tuning, +5.4 over the reference) and 71.90% (LoRA, +3.6 over the reference LoRA) on the CodaBench validation set.

2. Related Work

Vision Transformers and transfer learning. ViT [5] applies a Transformer [21] to a sequence of 16×16 image patches. Supervised pretraining on ImageNet-21k [4] with strong augmentation and regularization (AugReg [19]) produces backbones that transfer well. We use the AugReg ViT-B/16 checkpoint from timm [22] that was further fine-tuned on ImageNet-1k. Better ImageNet models also transfer better, under both linear evaluation and fine-tuning [10]. Standard fine-tuning recipes use AdamW [15], warmup with cosine decay [14], label smoothing [20], stochastic depth [8] and layer-wise learning-rate decay [1]; our full fine-tuning baseline follows this practice.

Parameter-efficient fine-tuning (PEFT). Adapters insert small bottleneck MLPs into every Transformer block [6]. AdaptFormer [3] instead adds a scaled bottleneck branch *in parallel* to the MLP of each ViT block. LoRA [7] learns a low-rank update $\Delta W = BA$ to frozen weight matrices, which can be merged into the weights after training. VPT [9] prepends learnable prompt tokens, and BitFit [2] trains only the bias terms. A classic alternative is to fine-tune only the top layers, whose features are the most task-specific [23]. We implement LoRA with the PEFT library [16] and compare it with Adapter, AdaptFormer, BitFit and top-block fine-tuning under one protocol.

Initializing the classifier head. When a pretrained model is transferred to new classes, the head is usually re-initialized; ViT zero-initializes it [5]. LP-FT [12] argues that a randomly initialized head distorts pretrained features during fine-tuning. It therefore first fits a linear probe on frozen features and then fine-tunes everything. Prototype-based classifiers represent each class by the mean embedding of its examples, as in prototypical networks [18]. Such normalized class means can be *imprinted* directly as the weights of a cosine classifier [17]. Our prototype head is an imprinted cosine classifier with a fitted temperature. We compare it with random and linear-probe initialization in a regime that is many-way (10,000 classes) and few-shot per class (45 images), with a short training budget.

3. Method

Figure 1 summarizes the pipeline. Let f_θ map an image x to its final-layer `[CLS]` feature $z = f_\theta(x) \in \mathbb{R}^{768}$, and let $h(z) = Wz + b$ with $W \in \mathbb{R}^{10000 \times 768}$ be the new classifier head.

3.1. Backbone, budget and full fine-tuning

Backbone. We use the timm [22] checkpoint `vit_base_patch16_224.augreg2_in21k_ft_in1k`. It is ViT-B/16 pretrained on ImageNet-21k with AugReg [19] and fine-tuned on ImageNet-1k. The backbone has 85,798,656 parameters; the 1,000-class head is replaced by a new 10,000-class head with 7,690,000 parameters.

Compute budget. The project allows $2\,\text{h} \times 48\,\text{GB} = 96\,\text{GB·h}$ per training run ($\pm 10\%$). We convert this by the memory of the card we use: 1.2 h on an 80 GB A100. Our clock counts everything that uses the GPU for training: head initialization (including feature extraction), all optimizer steps, and the holdout evaluation after every epoch. It excludes unpacking the data, writing checkpoints and the final predictions. After timing training steps 20–120, the cosine schedule is compressed if the planned number of epochs would exceed 97% of the budget, and training stops at $1.05\times$ the budget. Every run reported here finished within 1.06 GPU-hours.

Full fine-tuning. All backbone and head weights are trained with AdamW [15] at learning rate 10^{-4} and weight decay 0.05 (none on biases, normalization parameters, the class token and position embeddings). We use layer-wise learning-rate decay of 0.75 [1] and drop path 0.1 [8]. The learning rate warms up linearly (one epoch for runs of 6 or more epochs, 0.3 epochs for 3-epoch runs) and then follows a cosine decay to zero [14]. Other settings are batch size 256, bf16 mixed precision, gradient clipping at norm 1, and label smoothing 0.1 [20]. Training augmentation is RandomResizedCrop (scale 0.25–1) plus horizontal flip. For evaluation, images are resized to 248 pixels and center-cropped to 224.

3.2. Parameter-efficient and partial adaptation

LoRA. For a frozen linear layer $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA [7] computes

$$y = W_0 x + \tfrac{\alpha}{r} BAx, \tag{1}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trainable. B is initialized to zero, so training starts exactly from the pretrained model. We use $r = 8$ and $\alpha = 16$. timm fuses the query, key and value projections into a single `qkv` layer. LoRA on `qkv` therefore adapts all three projections, with the same parameter count (294,912, 0.344% of the backbone) as LoRA on separate query and value projections. Our final

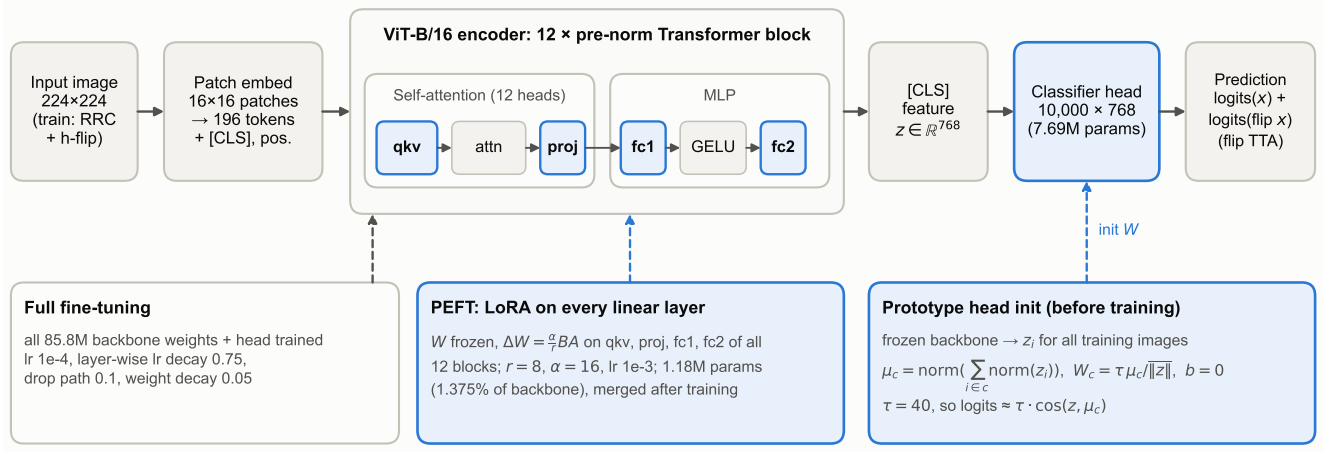


Figure 1. **Overview of our pipeline.** A ViT-B/16 backbone is adapted either by full fine-tuning or by LoRA on every linear layer (blue modules: qkv, proj, fc1, fc2 in all 12 blocks). Before training, the 10,000-way head is initialized by imprinting L2-normalized class-mean features (prototypes) with temperature τ. At test time, the logits of the image and of its horizontal flip are summed. All runs use AdamW with warmup and cosine decay, bf16 and batch size 256, within a budget of 96 GB·h (1.2 h on one A100-80GB).

PEFT variant applies LoRA to all four linear layers of every block (`qkv`, `proj`, `fc1`, `fc2`), for 1,179,648 parameters (1.375%). The LoRA weights and the head use learning rate 10^{-3} and weight decay 10^{-4}, with no LoRA dropout. After training, $\frac{\alpha}{r} BA$ is merged into W_0, so inference uses exactly the base architecture.

AdaptFormer. Following [3], each block gains a bottleneck branch in parallel to its MLP:

$$x_{l+1} = x' + \mathrm{MLP}(\mathrm{LN}(x')) + s \cdot W_{\mathrm{up}}\, \mathrm{ReLU}(W_{\mathrm{down}} x'), \quad (2)$$

where x' is the output of the attention sub-block. We use bottleneck width 64, scale $s = 0.1$, dropout 0.1 and a zero-initialized W_{up}, for 1,189,632 parameters (1.387%). Unlike LoRA, the branch cannot be merged into the backbone.

Adapter, BitFit and partial fine-tuning. For the comparison in Sec. 4.4 we add three more methods. The serial Adapter [6] applies the same bottleneck to the MLP output, $h + W_{\mathrm{up}}\, \mathrm{ReLU}(W_{\mathrm{down}} h)$ with $h = \mathrm{MLP}(\mathrm{LN}(x'))$, before the residual addition and without a scale. BitFit [2] trains only the bias vectors of all linear and normalization layers. Partial fine-tuning trains the last K blocks and the final normalization layer and freezes everything below them; one block has 7,087,872 parameters (8.26% of the backbone). Table 1 gives the trainable parameters of every method.

In all runs except the zero-shot baseline the head is also trained. Because the head alone has 7.69M parameters, we report trainable parameters both excluding it (the convention of the project's 0.34% LoRA reference) and including it.

Method	Count	Parameters	%
Partial, last K blocks	$7{,}087{,}872\,K + 1{,}536$	7.09M–42.5M	8.26–49.6
Full fine-tuning	all	85,798,656	100
BitFit	$12 \cdot 8{,}448 + 1{,}536$	102,912	0.120
Adapter, AdaptFormer	$12 \cdot 99{,}136$	1,189,632	1.387
LoRA, qkv only	$12 \cdot 3{,}072\,r$	294,912	0.344
LoRA, all linear	$12 \cdot 12{,}288\,r$	1,179,648	1.375

Table 1. **Trainable backbone parameters** (head excluded; $r = 8$; partial: $K \in \{1, 2, 4, 6\}$). 1,536: final LayerNorm (partial) or the patch-embedding and final-norm biases (BitFit); 8,448 biases per block. Adapter: $2 \cdot 768 \cdot 64 + 64 + 768 = 99{,}136$ per block. LoRA adds $r\,(d_{\mathrm{in}} + d_{\mathrm{out}})$ per layer, *i.e.* $3{,}072\,r$ for qkv and $12{,}288\,r$ for qkv, proj, fc1 and fc2 together.

3.3. Initializing the 10,000-way head

The head is the largest new component, and each of its rows sees at most 45 images. A random head produces near-uniform logits (initial loss $\approx \ln 10000 = 9.2$), and the backbone receives large, noisy gradients from it early in training. We compare three initializations.

Random. timm's default initialization of a new linear layer.

Prototype (imprinted) head. With the frozen pretrained backbone, we extract features z_i for all training images once. We then imprint L2-normalized class means [17, 18]:

$$\mu_c = \frac{\sum_{i:\, y_i = c} \hat{z}_i}{\left\| \sum_{i:\, y_i = c} \hat{z}_i \right\|}, \quad W_c = \frac{\tau}{\|z\|}\, \mu_c, \quad b_c = 0, \quad (3)$$

where $\hat{z}_i = z_i/\|z_i\|$ and $\overline{\|z\|}$ is the mean feature norm, so that logits are approximately $\tau \cos(z, \mu_c)$. The temperature τ is chosen from $\{5, 10, 15, 20, 30, 40, 60, 80\}$ by the cross-entropy on 10,000 training features, which selects $\tau = 40$. This head alone, with zero trainable parameters, reaches 44.0% holdout accuracy. Feature extraction takes 6–7 minutes on an A100, and this time is counted in every run's GPU-hours.

Linear-probe (LP) head. Starting from the prototype head, we train (W, b) on the cached frozen features for 10 epochs. We use AdamW (learning rate 10^{-3}, weight decay 10^{-4}), batch size 4,096, cosine decay and label smoothing 0.1. This reaches 62.1% holdout accuracy in a few seconds and corresponds to the first stage of LP-FT [12]. It was our initial design, chosen to avoid feature distortion from a random head. Section 4.5 shows that it is the worst of the three choices.

3.4. Test-time augmentation

Following common practice [11], we sum the logits of an image and its horizontal flip. This doubles inference cost and is applied only where stated.

4. Experiments

4.1. Setup

Data and protocol. Validation and test labels are hidden, so we hold out 2 images per class from the training split (20,000 images, fixed seed). We use this holdout for model selection and ablations, and train on the remaining 43 images per class. Final models are retrained on all 45 images per class. Validation and test images are used only to produce submission files. CodaBench reports top-1 accuracy. Because the validation set is balanced (5 images per class), top-1 accuracy equals the average per-class accuracy. Our holdout tracks the server closely: 72.31% *vs*. 72.57% for full fine-tuning and 71.98% *vs*. 71.90% for LoRA. Differences below about 0.3 points are within holdout noise. All runs use one seed.

Metrics. Latency is the mean time of 100 forward passes of a random 224×224 input at batch size 1, after 10 warm-up passes, with bf16 autocast on an A100. Throughput is measured at batch size 256. Measurements of the same architecture vary by up to about 0.5 ms between runs. GPU-hours are taken from the clock described in Sec. 3.1. Peak VRAM is the maximum memory allocated by PyTorch during training; the memory reserved by PyTorch is 0.2–1.4 GB higher (20.1 GB for the final model). The appendix contains the supporting material:

- training-log excerpts with time and VRAM (Appendix A);
- all 26 runs (Tab. 8);
- the full hyperparameters (Tab. 9).

Method	Trainable	Reference	Ours
Zero-shot, initialized head	0%	43.7	43.95
Full fine-tuning	100%	70.8	70.88
LoRA ($r = 8$)	0.34%	68.3	68.57

Table 2. **Reproducing the reference results** from the project description (top-1, %). Ours: holdout accuracy of 3-epoch runs (0.40–0.45 GPU-h) with the linear-probe head; for zero-shot, the prototype head without training.

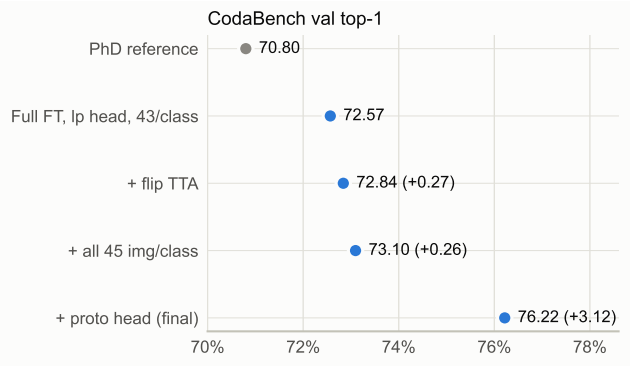


Figure 2. **CodaBench validation accuracy as we change one setting at a time** (all runs: full fine-tuning, 8 epochs). The gray dot is the reference. Positions encode values, so the axis starts at 70%.

Hardware and software. All runs use one NVIDIA A100-SXM4-80GB with 12 CPU cores, except one ablation batch (random and prototype heads for LoRA in Tab. 5), which ran on an RTX PRO 6000 Blackwell. Repeating its prototype run on the A100 gave 71.98% *vs*. 72.00%, so accuracies are comparable across the two GPUs; speeds and latencies are not. Software, as logged from batch `1019_1120` on, which covers all final models and the method comparison: PyTorch 2.11.0 (CUDA 13.0), timm 1.0.30 and PEFT 0.21.1. Earlier runs used the then-current releases, which were not logged.

4.2. Reproducing the reference baselines

Before changing anything, we checked our pipeline against the three reference results in the project description. Short 3-epoch runs reproduce all three within 0.3 points (Tab. 2). This confirms the data pipeline, the evaluation and our reading of the trainable-parameter convention.

4.3. Main results

Table 3 reports all four required metrics for the key models. Our final full fine-tuning model reaches **76.22%** on the CodaBench validation set, 5.4 points above the reference. It uses 1.03 GPU-hours, 18.9 GB of memory and the unchanged

Method	Head init	Ep.	Trainable params		Top-1 accuracy (%)			Inference		Training	
			Backbone	Head	Holdout	Val	Test	ms (bs 1)	img/s	GPU-h	GB
Zero-shot	prototype	0	0	0	43.95	–	–	7.55	3,832	0.11	3.9
Linear probe	LP	–	0	7.69M	62.07	–	–	–	–	≈0.11	–
Full fine-tuning	LP	8	85.80M (100%)	7.69M	72.31	72.57	–	7.35	3,864	1.03	18.9
Full fine-tuning (final)†	prototype	8	85.80M (100%)	7.69M	–	**76.22**	TBD	14.95‡	1,919‡	1.03	18.9
LoRA, qkv only	LP	3	0.29M (0.344%)	7.69M	68.57	–	–	7.22	3,817	0.40	13.5
AdaptFormer	LP	3	1.19M (1.387%)	7.69M	69.05	–	–	9.89	3,461	0.39	12.8
LoRA, all linear (final)	prototype	3	1.18M (1.375%)	7.69M	**71.98**	**71.90**	TBD	7.61	3,869	0.54	19.9

Table 3. **Main results.** Holdout: 2 images per class held out from the training split; Val/Test: CodaBench validation/test (Phase 2). Backbone trainable parameters are also given as a fraction of the 85.8M backbone; the 10,000-way head (7.69M) is trained in every method except zero-shot. Latency, throughput, GPU-hours and peak VRAM are measured on one A100-SXM4-80GB; LoRA is evaluated after merging. †Trained on all 45 images per class (no holdout). ‡With flip TTA (two forward passes); without TTA the architecture is identical to the row above it (7.35 ms).

Prototype head, 3 epochs	Params	Holdout	img/s	GB	ms
Linear probe	0	61.94	–	3.2	6.96
Partial, $K = 1$	8.26%	67.97	1,706	3.8*	7.92
Partial, $K = 2$	16.5%	70.99	1,655	4.1	7.41
Partial, $K = 4$†	33.0%	71.88	1,611	7.0	7.41
Partial, $K = 6$†	49.6%	72.89	1,650	10.0	7.42
Full fine-tuning	100%	**73.38**	1,139	18.9	7.78
BitFit	0.120%	69.81	1,407	12.7	7.33
Adapter	1.387%	71.54	1,509	12.8	9.74
AdaptFormer	1.387%	71.17	1,436	12.8	10.23
LoRA, all linear	1.375%	**71.98**	888	19.9	7.61

Table 4. **Adaptation methods under one protocol** (holdout top-1, %; A100). Params: trainable backbone parameters. img/s: training throughput; GB: peak training memory; ms: batch-1 latency, LoRA merged. *Peak during feature extraction; training used less. †Full fine-tuning recipe (Sec. 3.1); $K = 1, 2$ use learning rate 3×10^{-4} without layer decay or drop path. The linear probe was re-run for this comparison (62.07 in Tab. 3).

ViT-B/16 architecture. Figure 2 shows how we got there, one change at a time:

- flip TTA adds 0.27 points; it changes 4.6% of the predictions and doubles latency;
- training on all 45 images per class adds 0.26 points, which is at the edge of the noise but in the expected direction;
- replacing the LP head by the prototype head adds **3.12 points**.

Our final PEFT model, LoRA on all linear layers with a prototype head, reaches **71.90%** (+3.6 over the reference LoRA). It trains 1.375% of the backbone, and after merging it has exactly the latency of the base model. Under identical settings (3 epochs, prototype head, holdout), it attains 98% of full fine-tuning accuracy (71.98% *vs*. 73.38%). Phase 2: add CodaBench test accuracy of the final models.

4.4. Comparing adaptation methods

The project description lists several ways to adapt the backbone. To compare them fairly, we ran seven methods with one protocol: the prototype head (Sec. 3.3), 3 epochs, the same holdout split and the same GPU (Tab. 4 and Fig. 3). The efficient methods all use learning rate 10^{-3} and were not tuned individually.

Where the parameters sit matters more than how many there are. Methods that make small changes in every block are far more parameter-efficient than fine-tuning the top blocks. BitFit trains 0.12% of the backbone and beats fine-tuning the last block (8.3%) by 1.8 points; LoRA (1.375%) matches the last four blocks (33%), and the Adapter (1.387%) beats the last two (16.5%). Partial fine-tuning shows diminishing returns: the first block adds 6.0 points over the linear probe, the second 3.0, and each further pair about 1; six blocks come within 0.5 points of full fine-tuning. A plausible reason is that frozen lower blocks cannot adapt their features to the new classes, whereas spread-out changes reach every block. Among the efficient methods, LoRA is the most accurate (71.98%), ahead of the Adapter (71.54%), AdaptFormer (71.17%) and BitFit (69.81%). The serial Adapter and the parallel AdaptFormer differ by 0.37 points, about our noise level, and they also differ in scale (1.0 *vs*. 0.1), so we see no advantage of the parallel placement here. None of them reaches full fine-tuning (73.38%), although the project description expects parameter-efficient methods to outperform it on small data. One reason may be that, with 450,000 training images in total, the backbone itself is not data-starved, even if each class is.

Memory follows the depth of the backward pass. Training memory depends on how far gradients have to propagate, not on how many parameters are trained (Fig. 3, right). Partial fine-tuning adds about 1.5 GB per trained block (4.1, 7.0 and 10.0 GB for $K = 2, 4, 6$); extrapolated to all 12 blocks this gives 18.8 GB, which matches full fine-tuning

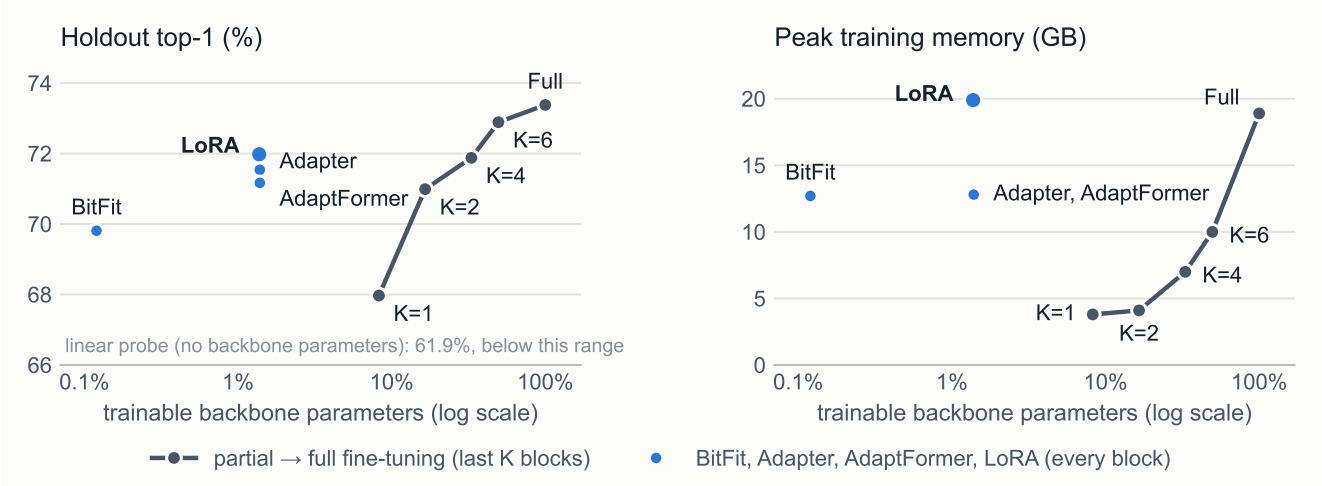


Figure 3. **Holdout accuracy (left) and peak training memory (right) against the share of trainable backbone parameters** (log scale) for the runs in Tab. 4. Methods that change every block (blue) are far more parameter-efficient than fine-tuning the last K blocks (gray), but they need more memory, because their gradients reach the first block. The linear probe (61.9%, no backbone parameters) lies below the left panel; the memory of $K = 1$ is the feature-extraction peak.

Head init	Head alone	LoRA all 3 ep holdout	AdaptF. 3 ep holdout	Full FT 3 ep holdout	Full FT 8 ep, TTA val
Random	chance	71.95*	–	–	–
Prototype	44.0	**72.00***	**71.17**	**73.38**	**76.22**
Linear probe (LP)	62.1	69.45	69.05	70.88	73.10

Table 5. **Head initialization** (top-1, %). Within each column only the head initialization changes. Head alone: accuracy of the initialized head before training. AdaptF.: AdaptFormer. Full FT 8 ep: all 45 images per class, flip TTA, CodaBench validation. *RTX PRO 6000; repeating the prototype run on the A100 gives 71.98.

(18.9 GB). BitFit and the two adapters train at most 1.4% of the backbone but need 12.7–12.8 GB, more than partial fine-tuning of half the network, because their gradients must reach the first block. They still use a third less memory than full fine-tuning and train 24–32% faster (1,407–1,509 *vs*. 1,139 img/s), since frozen weight matrices need no weight gradients. LoRA is the exception (19.9 GB, 888 img/s; Sec. 4.6). At inference, only the two adapters cost extra: 9.74 and 10.23 ms at batch size 1, against 7.33 ms for the unchanged architecture.

4.5. Ablation: classifier head initialization

Table 5 isolates the head initialization in four settings. In all four, the LP head is clearly worst, 2.1–3.1 points below the prototype head. Random and prototype initialization end up level for LoRA (71.95% *vs*. 72.00%). The prototype head, however, learns faster: it leads by 5 points after the first epoch (63.3% *vs*. 58.4%). Figure 4 shows the paradox: the LP head is 18 points better before training and worse after it.

We attribute the LP penalty to the probe having been fitted on *the same images that are then used for fine-tuning*. The probe already fits them well. Averaged over the first 100 steps of the LoRA runs, the training loss is 2.29 with the LP head, compared with 4.39 with the prototype head and 9.28 with a random head. The fine-tuning gradient reaching the backbone is therefore small and dominated by the few examples the probe gets wrong. Once the features start to move, the fitted head no longer matches them: in the LoRA run with the LP head, the training loss *rises* from 2.22 to 2.69 during the first epoch, before any image has been seen twice. Figure 5 points the same way. With identical full fine-tuning settings, the LP model reaches a lower training loss but a much lower validation accuracy, *i.e*., it memorizes more and generalizes less. The same holds in every pair of Tab. 5; for AdaptFormer, for example, the LP run ends at a training loss of 1.86 *vs*. 1.98 and a holdout accuracy of 69.05% *vs*. 71.17%. A prototype head encodes the class structure, which explains its faster start, without having been fitted to individual training images. This complements LP-FT [12], which motivates the probe as protection against feature distortion: when the probe is fitted on the fine-tuning images themselves, with only 43–45 images per class, it becomes a liability.

4.6. Ablation: where to put PEFT capacity

Table 6 compares PEFT variants with the LP head for 3 epochs. **Capacity matters more than the learning rate.** Halving the LoRA learning rate changes nothing (−0.12, within noise). Extending LoRA from `qkv` to all four linear layers, so that the MLP is also adapted, adds 0.88 points

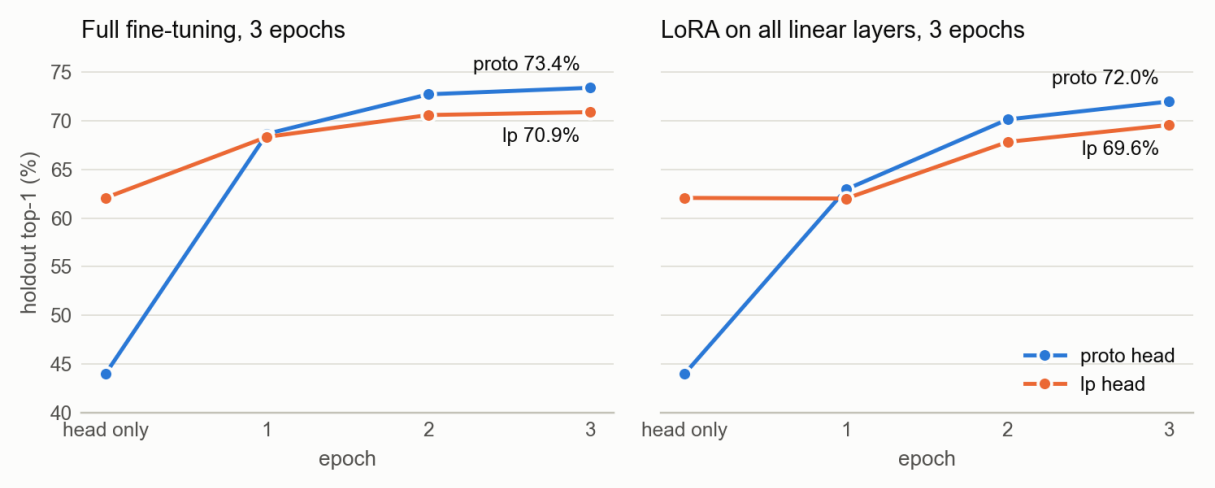


Figure 4. **Holdout accuracy during training for the LP and prototype heads** (3 epochs; left: full fine-tuning, right: LoRA on all linear layers). "head only" is the accuracy of the initialized head before any training. The LP head starts 18 points higher but finishes about 2.5 points lower.

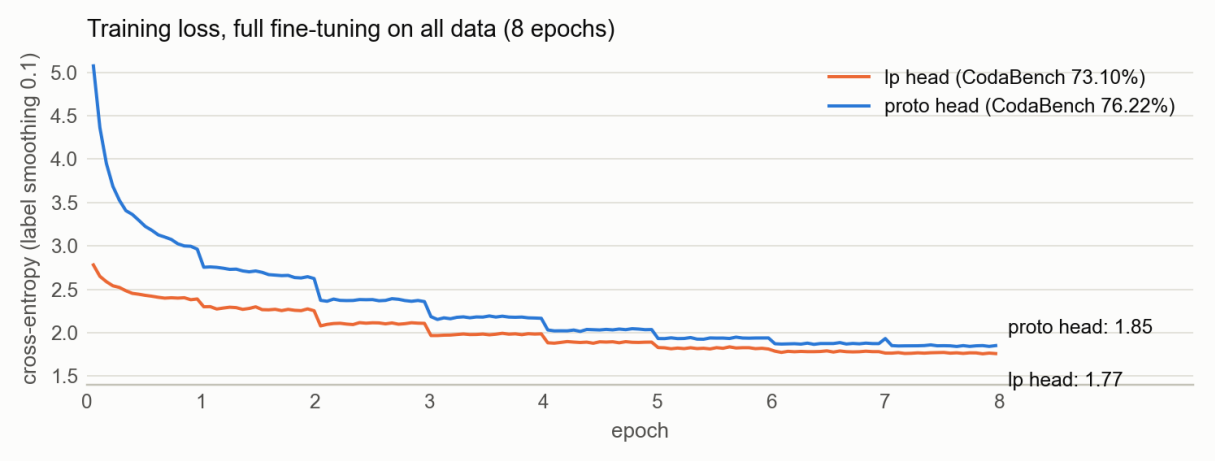


Figure 5. **Training loss of full fine-tuning on all data (8 epochs) with the LP and prototype heads**, which differ only in head initialization. The prototype model keeps a *higher* training loss (1.85 *vs*. 1.77 in the last epoch) but scores 3.12 points higher on the validation set. The loss drops at every epoch boundary and stays flat within an epoch, a sign that the model is memorizing the 45 images per class.

LP head, 3 epochs	Params	Holdout	img/s	GB	ms
LoRA qkv, lr 10^{-3}	0.344%	68.57	1,341	13.5	7.22
LoRA qkv, lr 5×10^{-4}	0.344%	68.45	1,353	13.5	7.15
LoRA all linear	1.375%	**69.45**	885	19.9	7.03
AdaptFormer	1.387%	69.05	1,437	12.8	9.89
Full fine-tuning	100%	70.88	1,131	18.9	7.31

Table 6. **PEFT design.** Params: trainable backbone parameters (head excluded). img/s: training throughput; GB: peak training memory; ms: batch-1 latency after merging (A100, bf16).

while staying far below the 5% that the project description gives as an example of a small set of parameters. **Mergeable updates matter for deployment.** AdaptFormer is close in accuracy, but its parallel branches cannot be merged and raise batch-1 latency from about 7.3 ms to 9.89 ms (+35%). **LoRA is not cheaper to train here.** LoRA on all linear layers trains *slower* than full fine-tuning (885 *vs*. 1,131 img/s) and peaks slightly higher in memory (19.9 *vs*. 18.9 GB). This is presumably because of the 96 extra small, unfused matrix multiplications per forward pass and the activations they keep for the backward pass. Its savings are in trainable parameters, optimizer state and checkpoint size (35.5 MB, of which 30.8 MB is the head, *vs*. 374 MB). The adapters and BitFit, in contrast, train faster and with a third less memory than full fine-tuning (Sec. 4.4).

Setting	Short (ep)	Long (ep)	Δ
Full FT, LP	70.88 (3)	72.31 (8)	+1.43
Full FT, prototype, final‡	74.01 (3)	76.22 (8)	+2.21
LoRA qkv, LP†	68.57 (3)	68.69 (10)	+0.12
LoRA all, LP	69.45 (3)	69.87 (6)	+0.42
LoRA all, prototype	71.98 (3)	71.52 (6)	−0.46

Table 7. **Training length** (holdout top-1, %; LoRA all: LoRA on all linear layers). ‡All 45 images per class with flip TTA, CodaBench validation. †The 10-epoch run also lowered the head learning rate to 2×10^{-4}.

4.7. Ablation: training length

The two adaptation styles spend the budget very differently (Tab. 7). Full fine-tuning keeps improving with longer schedules: going from 3 to 8 epochs adds 2.21 points for the final recipe. LoRA, in contrast, saturates after about 3 epochs, and with the prototype head 6 epochs are worse than 3. A plausible explanation is the learning rate. Full fine-tuning uses a small learning rate (10^{-4}) that is further reduced for early layers, while LoRA uses 10^{-3}. With only 43 images per class, the longer high-learning-rate phase mainly increases memorization. Its training loss falls much further in 6 epochs (1.70) than in 3 (1.95), while holdout accuracy falls too. Hence our final PEFT model uses 3 epochs (0.54 GPU-hours) and our final full fine-tuning model uses 8 epochs (1.03 GPU-hours), both within the 1.2-hour budget.

5. Conclusion

We adapted ViT-B/16 to a 10,000-class fine-grained benchmark with 45 images per class within a budget of 96 GB·h per run. The largest gain came from the least glamorous component. Initializing the 10,000-way head with imprinted class prototypes instead of a linear probe fitted on the training features improved accuracy by 2.1–3.1 points, for full fine-tuning, LoRA and AdaptFormer alike. With this head, full fine-tuning reaches 76.22% on the CodaBench validation set in 1.03 GPU-hours. LoRA on all linear layers reaches 71.90% while training 1.375% of the backbone. After merging, LoRA has zero inference overhead. Across seven adaptation methods run with one protocol, three choices mattered for parameter-efficient adaptation:

- where the trainable parameters go: small changes in every block beat fine-tuning whole top blocks with 12–69× fewer parameters, and LoRA works best on all linear layers, not only attention;
- whether the update can be merged (LoRA and BitFit *vs.* the two adapters);
- a short schedule (about 3 epochs).

Training memory, finally, follows how far gradients propagate rather than how many parameters are trained, so fine-tuning only the top blocks is the cheapest way to train, while spreading a few parameters over all blocks is the most accurate per parameter.

Limitations. Every configuration was trained once (seed 0), and the holdout split has a noise level of about ±0.3 points, so small differences such as the 0.26-point gain from the two extra images per class should be read with care. The dataset is anonymized, so we cannot relate our findings to its domain. One ablation batch ran on a different GPU; its accuracies are comparable, as checked by repeating one run, but its speeds and latencies are not. Our explanation of why the linear-probe head hurts is consistent with the loss curves but is not a controlled proof. The efficient methods share one learning rate that was not tuned per method, partial fine-tuning with $K = 4, 6$ uses a different recipe from $K = 1, 2$, and the Adapter and AdaptFormer also differ in scale, so these comparisons are not single-factor. We did not explore prompt-based methods such as VPT [9] or prefix tuning [13], higher input resolution, or larger budgets. Phase 2: report the CodaBench test accuracy of the final models. Check the course page limit (the CVPR limit is 8 pages excluding references).

References

[1] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations (ICLR)*, 2022. 2

[2] Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. 2, 3

[3] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. AdaptFormer: Adapting vision transformers for scalable visual recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 1, 2, 3

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009. 2

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021. 1, 2

[6] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019. 1, 2, 3

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 2

[8] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, 2016. 2

[9] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision (ECCV)*, 2022. 2, 8

[10] Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. 2

[11] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012. 4

[12] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022. 1, 2, 4, 6

[13] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021. 8

[14] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. 2

[15] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 2

[16] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022. 2

[17] Hang Qi, Matthew Brown, and David G. Lowe. Low-shot learning with imprinted weights. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 1, 2, 3

[18] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2, 3

[19] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? Data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research (TMLR)*, 2022. 1, 2

[20] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 2

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. 2

[22] Ross Wightman. PyTorch image models. https://github.com/huggingface/pytorch-image-models, 2019. 2

[23] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. 2

A. Training-log excerpts

The project asks for the training time and VRAM to be documented with training logs. Below are excerpts of `train_log.txt` for our two final models; the full logs (every 100 steps) are available on request. Dates are removed from the timestamps, Google Drive paths are shortened, and long lines are wrapped. GPU-hours include head initialization (feature extraction), training and per-epoch holdout evaluation (Sec. 3.1).

Final full fine-tuning model (prototype head, all 45 images per class, 8 epochs, flip TTA at prediction):

```
========== 1019_1319_1_full ==========
[13:32:14] full: lr 0.0001, head lr 0.0001, wd 0.05, layer decay 0.75, epochs 8, holdout 0/class,
           head init proto, time budget 1.20 h
[13:32:14] Environment: NVIDIA A100-SXM4-80GB (80 GB), bfloat16, 12 CPUs |
           torch 2.11.0+cu130 (CUDA 13.0), timm 1.0.30, peft 0.21.1
[13:38:32] Extracted features for 450,000 train / 0 holdout images in 6.1 min
[13:38:32] Head initialized from proto (6.2 min incl. feature extraction, counted in this run's GPU time)
[13:38:32] Trainable (excl. head): 85,798,656 = 100.000% of backbone | head 7,690,000 params (fine-tuned)
[13:38:32] Training up to 8 epochs x 1757 steps (batch 256x1, warmup 1757 steps)
[13:40:03] Speed 1146 img/s, 6.5 min/epoch
[13:46:13] == epoch 1: train loss 3.4158, 13.8 min used
[13:52:55] == epoch 2: train loss 2.6899, 20.5 min used
[13:59:51] == epoch 3: train loss 2.3729, 27.4 min used
[14:06:42] == epoch 4: train loss 2.1758, 34.2 min used
[14:13:40] == epoch 5: train loss 2.0321, 41.1 min used
[14:20:31] == epoch 6: train loss 1.9367, 47.9 min used
[14:27:32] == epoch 7: train loss 1.8741, 54.9 min used
[14:34:26] == epoch 8: train loss 1.8491, 61.8 min used
[14:34:29] Training done: 14056 steps, 1.03 GPU-hours (budget 1.20 h)
[14:34:29] Peak VRAM: 18.9 GB allocated, 20.1 GB reserved by PyTorch (GPU has 80 GB)
[14:34:32] Inference latency (batch 1): 14.95 ms/img | throughput (batch 256): 1919 img/s |
           NVIDIA A100-SXM4-80GB, bfloat16, with flip TTA
[14:35:29] [val] 50,000 predictions over 9,992 classes (max 13 per class) ->
           .../submission_val_1019_1319_1_full.zip
```

Final PEFT model (LoRA on all linear layers, prototype head, 3 epochs, 2 images per class held out):

```
========== 1019_1319_3_lora ==========
[14:57:51] lora: lr 0.001, head lr 0.001, wd 0.0001, layer decay 1.0, epochs 3, holdout 2/class, head init proto,
           time budget 1.20 h
[14:57:51] Environment: NVIDIA A100-SXM4-80GB (80 GB), bfloat16, 12 CPUs |
           torch 2.11.0+cu130 (CUDA 13.0), timm 1.0.30, peft 0.21.1
[15:06:07] Extracted features for 430,000 train / 20,000 holdout images in 7.2 min
[15:06:08] Prototype head (zero-shot, 0 trainable params): holdout top-1 44.01%
[15:06:08] Head initialized from proto (7.2 min incl. feature extraction, counted in this run's GPU time)
[15:07:15] Trainable (excl. head): 1,179,648 = 1.375% of backbone | head 7,690,000 params (fine-tuned)
[15:07:15] Training up to 3 epochs x 1679 steps (batch 256x1, warmup 503 steps)
[15:07:53] Speed 888 img/s, 8.1 min/epoch
[15:15:44] == epoch 1: train loss 3.2174, holdout top-1 62.94%, 15.7 min used
[15:24:12] == epoch 2: train loss 2.4125, holdout top-1 70.12%, 24.2 min used
[15:32:40] == epoch 3: train loss 1.9496, holdout top-1 71.97%, 32.6 min used
[15:32:41] Training done: 5037 steps, 0.54 GPU-hours (budget 1.20 h)
[15:32:41] Peak VRAM: 19.9 GB allocated, 21.3 GB reserved by PyTorch (GPU has 80 GB)
[15:33:00] Final holdout: top-1 71.98%, top-5 89.07%
[15:33:03] Inference latency (batch 1): 7.61 ms/img | throughput (batch 256): 3869 img/s |
           NVIDIA A100-SXM4-80GB, bfloat16, no flip TTA
[15:33:55] [val] 50,000 predictions over 9,991 classes (max 15 per class) ->
           .../submission_val_1019_1319_3_lora.zip
```

B. All runs

Run	Method	Head	Ep.	Trainable	Holdout	Val	GPU-h	GB	ms
0928_0636_1_linear	Zero-shot	prototype	0	0	43.95	–	0.11	3.9	7.55
0928_0636_3_full	Full FT	LP	3	100%	70.88	–	0.45	18.9	7.31
0928_0748_1_full	Full FT	LP	8	100%	72.31	72.57/72.84‡	1.03	18.9	7.35
1018_2306_1_full	Full FT, all data	LP	8	100%	–	73.10‡	1.03	18.9	14.92‡
1019_1120_1_full	Full FT	prototype	3	100%	73.38	–	0.48	18.9	7.78
1019_1319_2_full	Full FT, all data	prototype	3	100%	–	74.01‡	0.44	18.9	15.24‡
1019_1319_1_full	**Full FT, all data (final)**	prototype	8	100%	–	**76.22**‡	1.03	18.9	14.95‡
0928_0636_2_lora	LoRA qkv	LP	3	0.344%	68.57	–	0.40	13.5	7.22
0928_0748_2_lora	LoRA qkv, head lr 2×10^{-4}	LP	10	0.344%	68.69	–	1.06	13.5	7.53
1018_1239_1_lora	LoRA qkv, lr 5×10^{-4}	LP	3	0.344%	68.45	–	0.41	13.5	7.15
1018_1239_3_adaptformer	AdaptFormer	LP	3	1.387%	69.05	–	0.39	12.8	9.89
1018_1239_2_lora	LoRA all	LP	3	1.375%	69.45	–	0.55	19.9	7.03
1018_2306_2_lora	LoRA all	LP	6	1.375%	69.87	–	0.97	19.9	7.31
1019_0953_1_lora*	LoRA all	random	3	1.375%	71.95	–	0.37*	19.9	1.81*
1019_0953_2_lora*	LoRA all	prototype	3	1.375%	72.00	–	0.44*	19.9	1.82*
1019_1120_2_lora	LoRA all	prototype	6	1.375%	71.52	–	0.97	19.9	7.76
1019_1319_3_lora	**LoRA all (final PEFT)**	prototype	3	1.375%	**71.98**	**71.90**	0.54	19.9	7.61
1102_1149_1_linear_proto	Zero-shot (re-run)	prototype	0	0	43.95	–	0.10	3.8	7.32
1102_1149_2_linear_lp	Linear probe	LP	–	0	61.94	–	0.10	3.2	6.96
1102_1942_1_partial_k1	Partial FT, $K = 1$	prototype	3	8.263%	67.97	–	0.35	3.8	7.92
1102_1942_2_partial_k2	Partial FT, $K = 2$	prototype	3	16.524%	70.99	–	0.35	4.1	7.41
1102_1942_3_partial_k4	Partial FT, $K = 4$	prototype	3	33.046%	71.88	–	0.35	7.0	7.41
1102_1942_4_partial_k6	Partial FT, $K = 6$	prototype	3	49.568%	72.89	–	0.35	10.0	7.42
1102_2023_1_bitfit	BitFit	prototype	3	0.120%	69.81	–	0.39	12.7	7.33
1102_2017_1_adapter	Adapter	prototype	3	1.387%	71.54	–	0.37	12.8	9.74
1102_2212_1_adaptformer	AdaptFormer	prototype	3	1.387%	71.17	–	0.39	12.8	10.23

Table 8. **All 26 runs.** IDs encode the batch start time (month, day, hour, minute) and the position in the batch; the last block is the method comparison (Sec. 4.4). LoRA all: LoRA on all linear layers. Trainable: backbone parameters without the 7.69M head. GB: peak allocated training memory ($K = 1$: feature extraction). ms: batch-1 latency. ‡Flip TTA. *RTX PRO 6000 Blackwell; all other runs on an A100-SXM4-80GB (speed and latency are not comparable). The LP head (62.07%) is computed inside every LP run; 1102_1149_2 re-ran it alone.

C. Hyperparameters

	Full fine-tuning	LoRA	AdaptFormer	Linear-probe head
Trainable	all weights + head	A, B in qkv, proj, fc1, fc2 (all 12 blocks) + head	parallel MLP adapters + head	head only, on cached features
Method-specific	layer-wise lr decay 0.75, drop path 0.1	$r = 8$, $\alpha = 16$, no dropout, merged after training	bottleneck 64, scale 0.1, dropout 0.1	initialized from the prototype head
Optimizer	AdamW	AdamW	AdamW	AdamW
Learning rate	10^{-4} (head 10^{-4})	10^{-3} (head 10^{-3})	10^{-3} (head 10^{-3})	10^{-3}
Weight decay	0.05	10^{-4}	10^{-4}	10^{-4}
Schedule	linear warmup (1 epoch; 0.3 epochs for 3-epoch runs), then cosine decay to 0			cosine, no warmup
Epochs	3 or 8	3, 6 or 10	3	10
Batch size	256	256	256	4,096
Precision	bf16	bf16	bf16	fp32 head, fp16 features
Regularization	label smoothing 0.1, gradient clipping at norm 1.0			label smoothing 0.1
Augmentation	RandomResizedCrop (scale 0.25–1) + horizontal flip			none (center crops)

Table 9. **Hyperparameters.** Weight decay is not applied to biases, normalization parameters, the class token and position embeddings. Evaluation resizes to 248 pixels and takes a 224 center crop.

Methods added for Sec. 4.4 train for 3 epochs with the settings above, except: partial fine-tuning uses learning rate 3×10^{-4} and weight decay 0.05 for $K = 1, 2$ and the full fine-tuning settings for $K = 4, 6$ (drop path only in trained blocks); BitFit uses learning rate 10^{-3} and no weight decay; the Adapter is AdaptFormer's bottleneck placed after the MLP, with scale 1.0.